Secure Coding Review of a Bank Management System Using C++

CodeAlpha Cyber Security Internship

Prepared By

Abdul Wasay

Date

June 2026

---

1. Introduction

The purpose of this Secure Coding Review is to identify security vulnerabilities and coding weaknesses within a Bank Management System developed in C++. Secure coding practices are essential for protecting sensitive financial information and preventing unauthorized access. This review evaluates the application, identifies potential risks, and provides recommendations to improve the overall security of the system.

---

2. Application Overview

The Bank Management System is a console-based application developed in C++. The application allows users to perform basic banking operations such as login authentication, balance inquiry, deposits, and withdrawals.

Features

- User Login Authentication
- Deposit Money
- Withdraw Money
- Check Account Balance
- Exit Application

Because the application processes financial information, secure coding practices are critical to prevent misuse, fraud, and unauthorized access.

---

3. Review Methodology

The security review was conducted through manual code inspection and analysis of application logic.

The following areas were examined:

- Authentication Mechanisms
- Password Security
- Input Validation
- Transaction Processing
- Access Control
- Error Handling
- Security Best Practices

The objective was to identify vulnerabilities that could be exploited by malicious users and recommend mitigation strategies.

---

4. Security Findings

Finding 1: Hardcoded Credentials

Severity

High

Description

The application stores user credentials directly within the source code.

Example:

```
username = "admin";
password = "12345";
```

Risk

Hardcoded credentials expose sensitive information to anyone with access to the source code. Attackers can easily obtain valid login credentials and gain unauthorized access.

Recommendation

- Store passwords securely using cryptographic hashing algorithms.
- Avoid storing credentials directly in source code.
- Use secure databases or protected configuration files.

Finding 2: Lack of Account Lockout Mechanism

Severity

High

Description

The application allows unlimited login attempts without any restrictions.

Risk

Attackers can repeatedly attempt different passwords until the correct password is discovered through brute-force attacks.

Recommendation

- Limit login attempts to three or five tries.
- Temporarily lock accounts after multiple failed attempts.
- Implement CAPTCHA and Multi-Factor Authentication (MFA).

Finding 3: Missing Balance Validation

Severity

Medium

Description

The withdrawal function does not verify whether sufficient funds are available before processing transactions.

Risk

Users may withdraw amounts greater than their available balance, resulting in negative balances and financial inconsistencies.

Recommendation

Validate withdrawal requests before processing.

Example:

```
if(amount <= balance)
{
    balance -= amount;
}
else
{
    cout << "Insufficient Balance";
}
```

Finding 4: Negative Deposit Values Accepted

Severity

Medium

Description

The deposit function accepts negative values as valid deposits.

Risk

Negative deposits can manipulate account balances and create inaccurate financial records.

Recommendation

Validate all user inputs and only allow positive deposit amounts.

Example:

```
if(amount > 0)
{
    balance += amount;
}
else
{
    cout << "Invalid Deposit Amount";
}
```

Finding 5: Weak Password Policy

Severity

Medium

Description

The application uses a weak password value:

```
12345
```

Risk

Weak passwords can be easily guessed through brute-force and dictionary attacks.

Recommendation

Implement strong password policies:

- Minimum 8–12 characters
- Uppercase letters
- Lowercase letters
- Numbers
- Special characters

Example:

```
Abdul@2026Secure
```

---

5. Security Recommendations

The following measures should be implemented to improve the security of the application:

1. Remove hardcoded credentials.
2. Store passwords using secure hashing algorithms.
3. Implement Multi-Factor Authentication (MFA).
4. Enforce strong password policies.
5. Limit login attempts and lock accounts after repeated failures.
6. Validate all user inputs.
7. Verify sufficient account balance before withdrawals.
8. Maintain secure logging and monitoring mechanisms.
9. Conduct regular security assessments and code reviews.

---

6. Conclusion

This Secure Coding Review identified several vulnerabilities within the Bank Management System, including hardcoded credentials, lack of account lockout protection, weak password policies, and insufficient input validation. Addressing these issues will significantly improve the security, reliability, and resilience of the application. Implementing secure coding practices and conducting regular security reviews are essential steps toward protecting sensitive financial information and reducing cybersecurity risks.